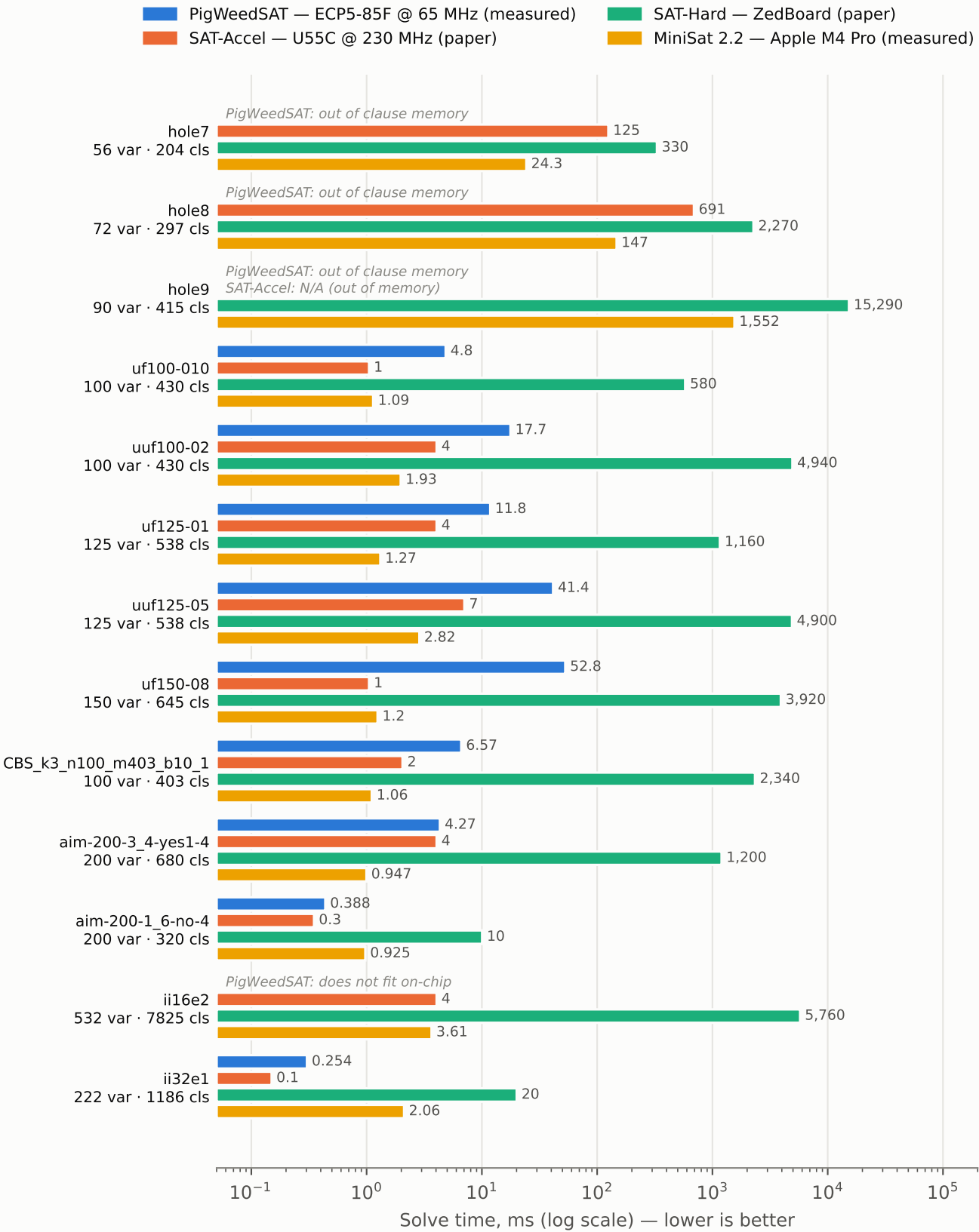




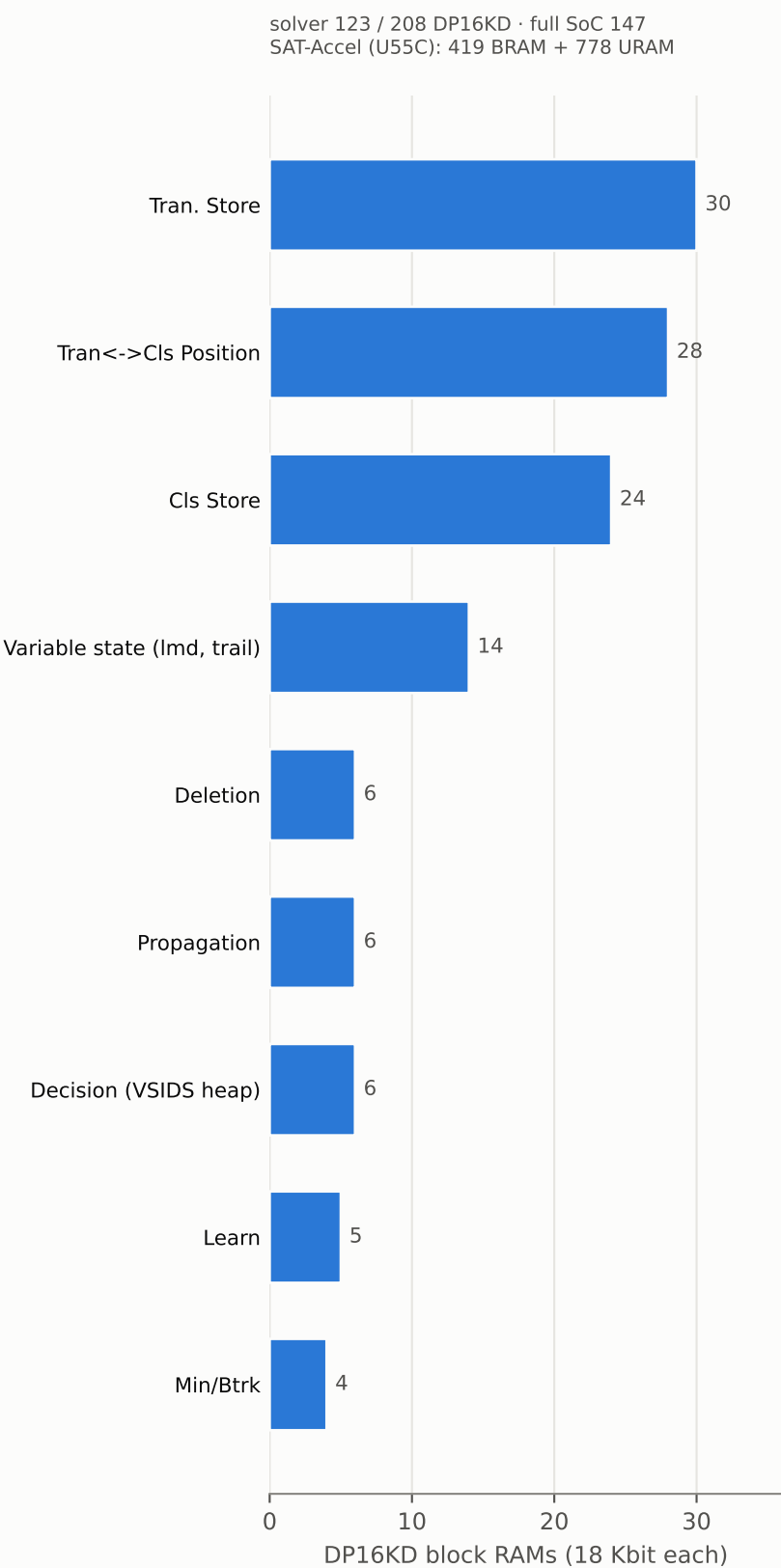
From the seed of the SAT-Accel grows PigWeed (SAT).

An Amaranth HDL implementation of SAT-Accel (Lo, Chang & Cong, FPGA '25) on a Lattice ECP5-85F, measured on Manhattan Reasoning cloud FPGAs.

(a) Table 3 — SATLIB instances used by SAT-Hard



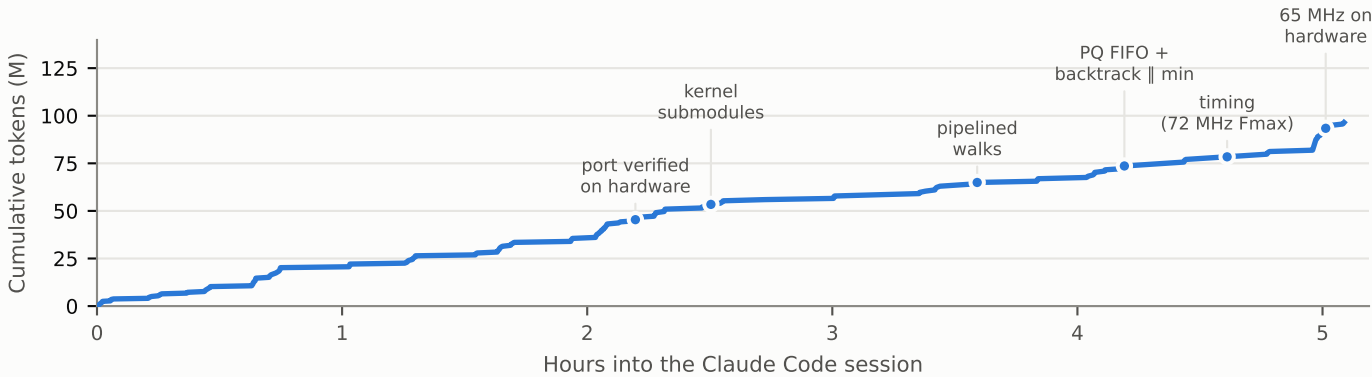
(b) Table 2 — block RAM by module



(c) Platforms

|                        | Board                | FPGA              | Node  | Clock   | LUTs  | FFs   | Block RAM                | DSPs  | Board DRAM  | Board cost |
|------------------------|----------------------|-------------------|-------|---------|-------|-------|--------------------------|-------|-------------|------------|
| PigWeedSAT (this work) | Lattice ECP5-5G Eval | ECP5 LFE5UM5G-85F | 40 nm | 65 MHz  | 84K   | 84K   | 208 × 18 Kb (3.7 Mb)     | 156   | —           | ~\$140     |
| SAT-Accel (FPGA '25)   | AMD Alveo U55C       | Virtex US+ XCU55P | 16 nm | 230 MHz | 1.30M | 2.61M | 2,016 × 36 Kb + 960 URAM | 9,024 | 16 GB HBM2  | ~\$5,400   |
| SAT-Hard (DSD '19)     | Digilent ZedBoard    | Zynq-7000 XC7Z020 | 28 nm | n/r     | 53K   | 106K  | 140 × 36 Kb (4.9 Mb)     | 220   | 512 MB DDR3 | ~\$620     |

(d) Compute spend — tokens used to build and measure PigWeedSAT



- 96.4M tokens in total
- 95.3M cache reads (99%): context re-read each turn
- 0.73M cache writes (new context)
- 0.38M output tokens (code, reasoning, tool calls)
- 186 model responses · claude-opus-5-5

PigWeedSAT: on-chip solve cycles at 65 MHz on a Manhattan Reasoning cloud ECP5-85F, host load excluded; all 9 solved answers correct and SAT models re-verified. Geometric mean over those 9: 122× faster than SAT-Hard, 3.9× slower than SAT-Accel, 4.1× slower than MiniSat on the M4 Pro. SAT-Accel and SAT-Hard times from Lo et al., FPGA '25, Table 3; SAT-Hard platform from Ustaoglu et al., DSD '19 (clock not reported, n/r). MiniSat 2.2: default options, median CPU time of 11 runs (includes parsing; ~1 ms floor). Board cost: mean of single-unit US list prices (Oct 2026), board only, no host. Compute spend: Claude Code session transcripts (tools/token\_log.py); tokens as reported by the API, not dollars.